

Bombberman RL: Neues Vorgehensmodell für Task 1 bis 4

Vom dokumentationsgetriebenen Softwareprojekt zu einem schlanken, wissenschaftlich belastbaren RL-Experimentierprozess.

Kernempfehlung: Ein lauffähiger Agent früh, eine Hypothese pro Experiment und eine Entscheidung pro Ergebnis. Dokumentation entsteht als Nebenprodukt der Experimente.

Auftrag

Die vorhandenen Issues, Projektregeln und die Milestone-Checkliste bleiben fachliche Grundlage. Ihre Granularität wird jedoch radikal reduziert, damit Agentenentwicklung, Training und belastbare Vergleiche wieder im Mittelpunkt stehen.

Stand: 17. August 2026

Geltungsbereich: Task 1 bis Task 4, beide geforderten Lernmodelle und Vorbereitung der Turnierauswahl

1. Management Summary

Die bisherige Projektgrundlage ist fachlich überwiegend sinnvoll, aber operativ falsch granularisiert. Die Milestone-Checkliste behandelt nahezu jede Datei, Hilfsfunktion, Metrik und Dokumentationsaktivität als eigenständiges Arbeitspaket. Dadurch entstehen lange Abhängigkeitsketten, bevor überhaupt ein einfacher gelernter Agent experimentell geprüft werden kann.

Die Kursvorgabe verlangt systematische Experimente, mindestens zwei unterschiedliche gelernte Modelle und einen nachvollziehbaren Vergleich. Sie verlangt kein Vorzeige-Softwareengineering-Projekt. Der Prozess sollte deshalb auf wenige vertikale Issues reduziert werden, die jeweils einen lauffähigen Agentenfortschritt oder eine wissenschaftliche Entscheidung liefern.

Zielbild: etwa 12 bis 14 substanzielle Issues für beide Modelle, Task 1 bis 4, Modellvergleich und Abgabe - statt 170 operativer Einzelpunkte.

Was sich konkret ändert

- Ein Issue entspricht einem vertikalen Agentenfortschritt, einem kontrollierten Experiment oder einer finalen Validierung.
- Einzelne Features, Reward-Werte, Tests, Konfigurationsdateien und Diagramme werden Acceptance Criteria - keine eigenen Issues.
- Task 1 liefert innerhalb weniger Tage einen vollständigen lernenden Coin Collector, nicht nur ein weiteres Scaffold.
- Jede Iteration verändert eine zentrale Variable und endet mit einer dokumentierten Entscheidung.
- Der zweite Modelltyp startet spätestens während Task 2, damit die Pflicht zu zwei Lernmodellen nicht zum Endrisiko wird.

2. Leitprinzipien des neuen Vorgehens

1. Agent vor Infrastruktur

Nur Infrastruktur bauen, die für den unmittelbar nächsten Trainings- oder Evaluationslauf benötigt wird.

2. Vertikale Slices statt Modul-Issues

Features, Lernalgorithmus, Rewards, Persistence, Tests und Smoke Run werden gemeinsam geliefert, sobald sie zusammen eine prüfbare Fähigkeit bilden.

3. Wissenschaftliche Entscheidungen statt Vollständigkeitschecklisten

Ein Experiment muss eine konkrete Frage beantworten. Daten, die keine Entscheidung beeinflussen, werden nicht erhoben.

4. Entwicklung schnell, Freeze belastbar

Zwischenvarianten werden mit kleinen, festen Budgets verglichen. Nur Milestone- und Finalkandidaten erhalten aufwendige Multi-Seed-Evaluationen.

3. Umgang mit den vorhandenen Issues

Im Repository bestehen aktuell drei offene Projekt-Issues. Sie bleiben Ausgangspunkt, werden aber anders zugeschnitten.

Issue #21 - Task-1-Q-Learning-Vertrag

- Abschließen, sofern der bestehende Pull Request ohne weitere Grundsatzdiskussion mergebar ist.
- Metriken, Seed-Trennung und Erfolgskriterium behalten.
- Keine weitere Verfeinerung, die die Implementierung blockiert.

Issue #22 - Agent Lifecycle dokumentieren

- Nicht als eigenständiges Gate weiterführen.
- Notwendige Lifecycle-Erkenntnisse kurz im Agenten-README oder in #23 festhalten.
- Danach als 'absorbed by #23' schließen.

Issue #23 - Baseline-Agentenpaket

- Vom Paketgerüst zum vollständigen End-to-End-Task-1-Agenten erweitern.
- Features, Q-Learning, Rewards, Save/Load, Tests, Training und Evaluation gemeinsam liefern.
- Ergebnis muss ein tatsächlich lernender Coin Collector sein.

Wichtige organisatorische Bereinigung

Die Nummern #1 bis #170 aus der PDF-Checkliste dürfen nicht mehr als Issue-IDs verwendet werden. Sie kollidieren bereits mit den echten GitHub-Nummern. Die Checkliste bleibt eine Referenz für mögliche Arbeitsschritte, nicht das operative Backlog.

4. Verdichtete Roadmap für Task 1 bis 4

Task	Implementierung	Experiment	Gate
1	End-to-End Q-Learning Coin Collector	State- und Reward-Varianten	Coins werden reproduzierbar gesammelt
2	Bomben, Crates und Flucht	Danger Features und Reward Shaping	Self-Kills sind kontrolliert
3	Opponent Features und Combat Rewards	Curriculum gegen Direkttraining	Gegnerjagd funktioniert
4	Multiplayer und Mixed Opponents	Eine zentrale Competitive-Hypothese	Agent 1 ist eingefroren

5. Task 1 - Q-Learning Coin Collector

T1-A - End-to-End Task-1 Q-Learning Agent

- Kompakte Coin- und Bewegungsfeatures inklusive stabilem State Key.
- Q-Tabelle, epsilon-greedy Exploration, Decay und Terminal-Update.
- Minimale sowie optional geformte Rewards.
- Relatives Save/Load, Training und deterministische Evaluation.
- Gezielte Tests für Encoder, Q-Update, gültige Aktionen und Persistence.
- Je ein Trainings- und Evaluations-Smoke-Test.

T1-B - State- und Reward-Varianten vergleichen

- Baseline: minimaler Zustand und minimale Rewards.
- Variante A: Coin-Richtung und Coin-Distanz.
- Variante B: zusätzlich Distance Reward Shaping.
- Drei Training-Seeds und identische ungesehene Evaluations-Seeds.
- Primärmetrik: gesammelte Coins bzw. Anteil gesammelter Coins.
- Diagnostik: Steps pro Coin und Invalid-Action-Rate.
- Ergebnis: eine begründet gewählte Task-1-Konfiguration.

Milestone-Gate

- Der Agent lernt sichtbar und übertrifft die vorher festgelegte Random Baseline.
- Training und Evaluation sind reproduzierbar; Seeds sind getrennt.
- Die ausgewählte Konfiguration ist durch ein kleines kontrolliertes Experiment begründet.

6. Task 2 - Bomben, Crates und Survival

T2-A - Bomben, Crates und Flucht integrieren

- BOMB-Aktion aktivieren.
- Blast-Reichweite und Explosionstiming korrekt abbilden.
- Kompakte Danger-, Escape- und Crate-Utility-Features ergänzen.
- Rewards für Crates, gefundene Coins, sichere Flucht und Self-Kills definieren.
- Blast-/Safety-Tests und Task-1-Regression ausführen.

T2-B - Danger Representation und Reward Shaping evaluieren

- Kleine Ablationsmatrix: Danger Features an/aus und Reward Shaping minimal/geformt.
- Gleiche Initialisierung, Seeds und Trainingsdauer.
- Primärmetrik: Coins pro Runde.
- Sicherheit: Self-Kill-Rate und Survival Rate.
- Effizienz: Crates pro Bombe.
- Ergebnis: Task-2-Checkpoint plus Task-1-Regressionwert.

Milestone-Gate

- Bomben-, Crate- und Fluchtverhalten funktionieren als zusammenhängende Lernaufgabe.
- Self-Kill-Rate und Survival sind unter identischen Bedingungen bewertet.
- Task-1-Navigation bleibt als Regression erhalten.

7. Task 3 - Opponent Hunting

T3-A - Gegnerjagd und Combat Learning integrieren

- Richtung und Distanz zum relevantesten Gegner.
- Erkennen, ob Gegner in Bombenreichweite gelangen können.
- Einfache Trap- und Escape-Information.
- Kill-, Death- und Suicide-Rewards.
- Curriculum von `peaceful_agent` zu `coin_collector_agent`.
- Regression auf Task 1 und 2.

T3-B - Curriculum gegen direktes Combat-Training

- Direktes Training gegen `coin_collector_agent` mit Curriculum vergleichen.
- Primärmetrik: Kill Rate.
- Diagnostik: Death Rate, Time-to-Kill und Score.
- Drei Training-Seeds mit identischem Evaluationsprotokoll.
- Ergebnis: eine gewählte Task-3-Konfiguration.

8. Task 4 - Competitive Multiplayer

T4-A - Competitive Multiplayer integrieren

- Mehrere Gegner kompakt priorisieren.
- Bomben- und Gegnergefahr kombiniert bewerten.
- Coin-versus-Combat-Kontext ergänzen.
- Training gegen einen gemischten Gegnerpool ermöglichen.
- Q-Table-Wachstum durch Diskretisierung oder Symmetrien begrenzen.
- Vollständige Task-1-bis-3-Regression.

T4-B - Eine zentrale Competitive-Hypothese testen

- Empfehlung: Mixed-Opponent-Training gegen reines Training mit `rule_based_agent` vergleichen.
- Nicht gleichzeitig Curriculum, Rewards, Features, Symmetrien und Hyperparameter ändern.
- Primär: mittlerer Turnier-Score.
- Zusätzlich: First-Place-, Kill-, Death- und Survival-Rate sowie `act()`-p95.

T4-C - Finalen Q-Learning-Agenten benchmarken und einfrieren

- Fünf Training-Seeds und ein größerer fester Evaluationslauf.
- Gegnerpool aus `random_agent`, `coin_collector_agent`, `rule_based_agent` und eigenen Checkpoints.
- Task-1-bis-4-Regressionsübersicht und repräsentativer Checkpoint.
- Laufzeit-, Self-Containment- und Kompatibilitätsprüfung.

9. Zweiter gelernter Modelltyp

Mindestens zwei unterschiedliche gelernte Modelle sind Pflicht. Der zweite Ansatz darf deshalb nicht erst nach Abschluss aller Q-Learning-Milestones beginnen.

Empfohlener Paralleltrack

- Ein- bis zweitägiger Spike: DQN gegen PPO anhand Implementierungsaufwand, vorhandener Hardware, erster Lernkurve und CPU-Inferenz vergleichen.
- Ausgewählten zweiten Agenten als einen End-to-End-Slice für Task 1/2 implementieren.
- In einem weiteren Issue auf Task 3/4 trainieren und sinnvoll tunen.
- Beide Modelle anschließend mit identischen Seeds, Gegnern, Runden und Metriken vergleichen.

PPO ist eine Option, aber keine unveränderliche Vorentscheidung. Ohne vorhandenen Gymnasium-Wrapper und stabile Trainingspipeline kann ein DQN mit diskretem Action Space der risikoärmere zweite Modelltyp sein. Der Spike entscheidet anhand von Evidenz.

Teamprinzip: Beide Modelle werden gemeinsam entworfen, implementiert und bewertet. Eine starre Aufteilung 'eine Person pro Modell' ist zu vermeiden.

10. Minimaler wissenschaftlicher Standard

Jedes Experiment-Issue enthält vor dem Training:

- Hypothese und genau eine zentrale geänderte Variable.
- Baseline und konstant gehaltene Bedingungen.
- Training-Seeds, Budget und Evaluationsprotokoll.
- Eine Primärmetrik, wenige diagnostische Metriken und eine Entscheidungsregel.

Nach dem Lauf werden nur Ergebnis, Interpretation und nächste Entscheidung ergänzt.

Zwei Evaluationsstufen

Schnelle Entwicklung

- Drei Training-Seeds und 100 bis 300 Evaluationsrunden pro Variante.
- Eine Primärmetrik und höchstens drei diagnostische Metriken.
- Keine aufwendige Statistik für jeden Zwischenstand.

Milestone-Freeze

- Fünf Training-Seeds und 500 bis 1.000 identisch konfigurierte Evaluationsrunden.
- Variation oder Bootstrap-Intervall und vorher festgelegte Auswahlregel.
- Checkpoint, Konfiguration und Git-Commit sichern.

11. Dokumentation, Tests und CI

Dokumentation entsteht nur noch an drei Stellen:

- Agenten-README: aktueller Modellstand, Features, Rewards, Training und Limitationen.
- Experimentordner: Config, Ergebnisdatei, eine relevante Abbildung und kurze Entscheidung.
- Abschlussbericht: zusammenhängende wissenschaftliche Darstellung.

Nummerierte Architektur- und Workflow-Dokumente werden nur dann aktualisiert, wenn sich eine dauerhafte Projektentscheidung ändert. Die Dokumentation darf kein Gate vor der Agentenimplementierung werden.

Tests mit hohem Informationswert

- Feature- und State-Encoding.
- Blast- und Danger-Berechnung.
- Q-Update inklusive Terminaltransition.
- Gültige Aktionen und Evaluation ohne Updates.
- Save/Load sowie je ein Training- und Evaluations-Smoke-Test.

Keine eigenen Contract-Tests für Markdown-Links, jedes Config-Feld oder jede kleine Hilfsfunktion. CI soll schnell Rückmeldung geben und Trainingsexperimente nicht durch Prozesskosten dominieren.

12. Realistischer Terminplan

Zeitraum	Fokus
17.-23. Aug.	Task 1 implementieren, trainieren und experimentell abschließen
24.-30. Aug.	Task 2; parallel Spike und Start des zweiten Modells
31. Aug.-5. Sept.	Task 3; zweites Modell auf Task 1/2
6.-11. Sept.	Task 4, Mixed-Opponent-Training, zweites Modell weiterführen
12.-16. Sept.	Cross-Model-Benchmark, Kandidat wählen, Submission-Test vorbereiten
17. Sept.	Optionaler offizieller Submission-Test
18.-20. Sept.	Nur kritische Bugfixes, Docker und Packaging
21. Sept.	Agent-Code abgeben
22.-28. Sept.	Bericht aus den Experimentaufzeichnungen erstellen

Ab dem 12. September zählt Auswahl und Robustheit mehr als neue Features. Nach dem optionalen Submission-Test werden nur noch kritische Fehler behoben.

13. Operativer Ziel-Backlog

Empfohlene Größenordnung für das gesamte verbleibende Projekt:

- Task 1 bis 4 für Q-Learning: acht Issues einschließlich des umgeschnittenen #23.
- Zweiter Modelltyp: zwei bis drei Issues plus kurzer Technologie-Spike.
- Cross-Model-Vergleich und Kandidatenauswahl: ein Issue.
- Finale Kompatibilität, Packaging und Abgabe: ein Issue.

Gesamt: etwa 12 bis 14 substanzielle Issues. Die 170 Punkte bleiben als Prüfliste im Hintergrund, nicht als operative Abhängigkeitskette.

14. Schlussfolgerung

Die bisherige Arbeit war nicht nutzlos: Repository-Struktur, Submission-Constraints, CI und Experimentgrundsätze bilden eine solide Basis. Problematisch ist allein, dass diese Basis zum Hauptprodukt geworden ist.

Die Kurskorrektur besteht nicht darin, wissenschaftliche Sorgfalt aufzugeben. Sie besteht darin, Sorgfalt dort einzusetzen, wo sie die Note und Turnierleistung tatsächlich verbessert: in trainierbaren Agenten, kontrollierten Vergleichen, verlässlichen Seeds, aussagekräftigen Metriken und nachvollziehbaren Entscheidungen.

Ab sofort sollte jedes Arbeitspaket die Frage beantworten: Liefert es innerhalb weniger Tage entweder eine neue lernbare Fähigkeit oder Evidenz für die nächste Designentscheidung? Wenn nicht, gehört es nicht in den kritischen Pfad.

Quellen und Projektgrundlagen

- Machine Learning Essentials 2026: Final Project - Reinforcement Learning for Bombberman, Kurs-Handout.
- Bombberman RL Milestone-Checkliste, Stand 17. August 2026.
- Repository-Dokumente README.md, CONTRIBUTING.md und docs/0001 bis docs/0006.
- GitHub-Issues #21, #22 und #23 im Repository 1BlauNitrox/mle-final-project.